Glossar :

- MVVM = Model-View-ViewModel (mit zentralem MessageHandler)

Vorteile meiner Architektur

- **Sprachübergreifende Zusammenarbeit** (Delphi, Python, C#, C++, Java)
- **Lose Kopplung** durch EventBus
- **Saubere Trennung** durch MessageHandler-Filter (nur `TDataModule`-Derivate)
- **Erweiterbarkeit** ohne Code-Brüche
- **Testbarkeit** durch isolierte Module
- **Team-Parallelität**: drei Teams, fünf Sprachen, ein gemeinsamer Bus

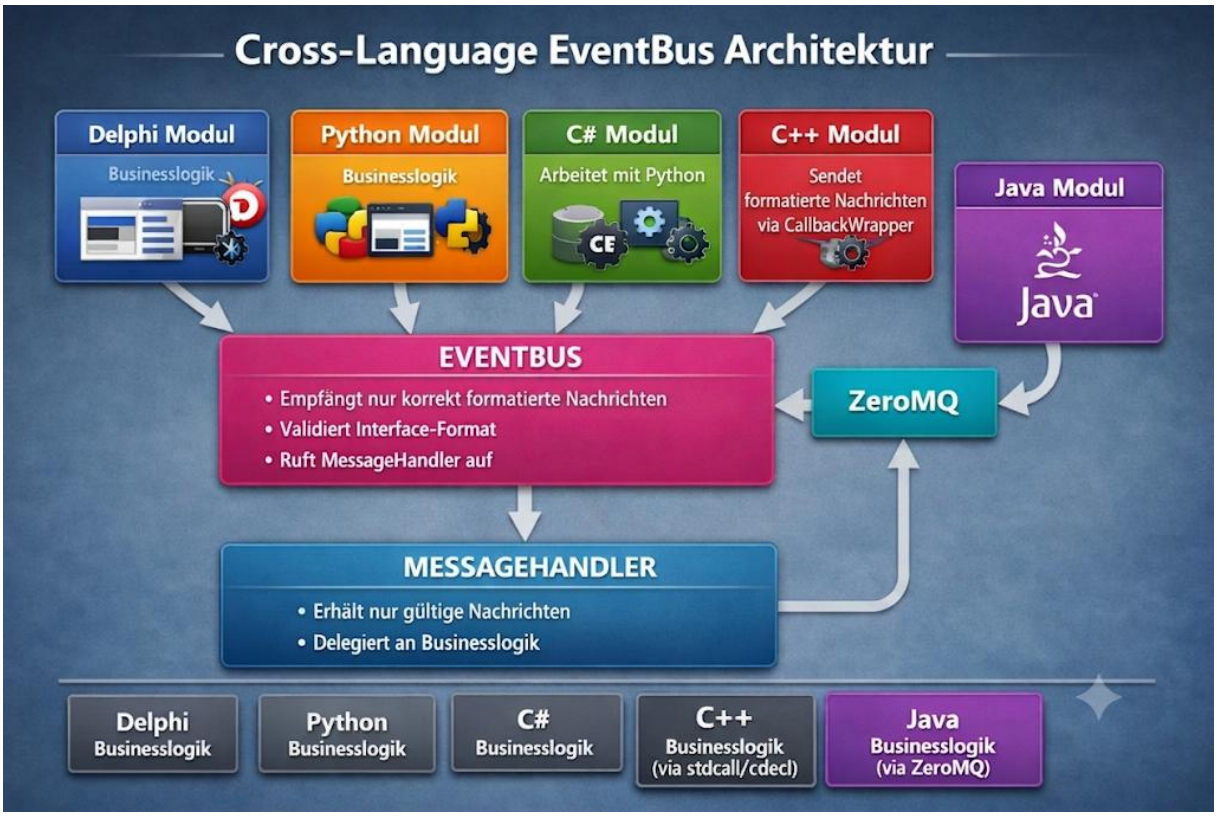
Sprache	Rolle	Integration
Delphi	Kern-Applikation, UI, EventBus, MessageHandler	Native
Python	Datenanalyse, KI-Logik, Skripting, Automatisierung	über Python4Delphi
C#	Business-Module, DLL-Logik, externe Services	über Python.NET-Interop
C++	Maschinensteuerung, Hersteller-SDKs, Echtzeit-Logik	DLL-Export stdcall/cdecl + CallbackWrapper zum EventBus
Java	Business-Module, externe Systeme, Enterprise-Integration	Externer Prozess via ZeroMQ + JSON-DTOs

Tool - Heisenberg

Was genau macht das Tool?

Das Tool ist ein modulares MVVM-System, in dem unabhängige Module über ein zentrales Message-System miteinander kommunizieren. Jedes Modul enthält eigene Business- und bei Bedarf Anzeigenlogik. Als Grundausstattung umfasst es Datenbank-, Dokumenten-, Webseiten- und Personen-/Mandantenverwaltungssysteme. Die Module können dabei, in einer

der 5 aufgeführten Technologien entwickelt werden. So können Teams mit unterschiedlichem KnowHow zu einem Team verbunden werden. Bis auf Java könnte alles in einer Applikation stattfinden.



Warum braucht der Kunde das Tool?

Das Tool ermöglicht es, schwer wartbaren Legacy-Code schrittweise zu modernisieren. Durch den Eventbus lässt sich neue Logik nahtlos in bestehende Systeme integrieren. So entsteht eine Brücke zwischen alter Architektur und moderner Modularität.

Wie genau kann der Kunde das Tool bei sich einsetzen / nutzen?

Der Kunde bindet den Eventbus in vorhandene Delphi-Datenmodule ein. Bestehende Funktionen können Events senden oder empfangen, ohne Codeabhängigkeiten. Neue Module – auch in Python, C#, C++ oder Java – lassen sich über die Event-Schnittstelle anbinden. C++-Hersteller-SDKs (Maschinensteuerungen) werden per stdcall/cdecl-Callback eingebunden; Java-Module kommunizieren über ZeroMQ als externen Prozess. Dadurch kann das System schrittweise erweitert und modernisiert werden. Die Integration erfolgt ohne Unterbrechung des laufenden Betriebs.

Was würde beim Kunden passieren, wenn er das Tool nicht einsetzt?

Ohne das Tool bleibt der Kunde in einer starren Legacy-Architektur gefangen, in der jede Änderung hohe Risiken und Kosten verursacht.
Module bleiben direkt miteinander verkoppelt, sodass jede Erweiterung

Seiteneffekte erzeugt und das System zunehmend schwer wartbar wird. Neue Technologien wie Python-Analyse, moderne C#-Module, C++-Maschinenanbindung, Java-Integration oder parallele Teamarbeit lassen sich kaum integrieren, wodurch Innovation ausgebremst wird.